

Lab 09

Animation in OpenGL

Objectives

- We will learn to create animation in OpenGL.

Programming Exercises

(90 minutes)

Task 1: Moving Objects (60 minutes)

Task 1A: Preparing the Smiley (20 minutes)

Let's first declare a `MyMovingSmiley` class. Part of `CGLab09.hpp` should have these lines below:

```
class MyMovingSmiley
{
public:
    MyMovingSmiley();
    ~MyMovingSmiley();

    void draw();
    void tickTime(long int elapsedTime);

private:
    GLUquadricObj *pObj;
    GLfloat posX, posY, posz;
    GLfloat roty;
};

//-----
//the main program will call methods from this class
class MyVirtualWorld
{
public:
    MyMovingSmiley mymovingsmiley;

    void draw()
    {
        drawFloor();
        mymovingsmiley.draw();
    }

    void drawFloor()
    {
        glDisable(GL_CULL_FACE);

        glColor3f(0.0f, 0.0f, 1.0f);
        glBegin(GL_QUADS);
            glVertex3f( 19.0f, 0.0f, 19.0f);
            glVertex3f( 19.0f, 0.0f,-19.0f);
            glVertex3f(-19.0f, 0.0f,-19.0f);
            glVertex3f(-19.0f, 0.0f, 19.0f);
        glEnd();
        glEnable(GL_CULL_FACE);
    }
}
```

```

//NOTE: different compare with those in earlier lab
void tickTime()
{
    //to fill later
}

//for any one-time only initialization of the
//    virtual world before any rendering takes place
//    BUT after OpenGL has been initialized
void init()
{
    //to fill later
}
};

```

The CGLab09.cpp should be as below:

```

/*
TCG6223 Computer Graphics

CGLab09.cpp

Objective: Lab09 on Animation
*/

#include <GL/glut.h>
#include "CGLab09.hpp"

using namespace CGLab09;

MyMovingSmiley::MyMovingSmiley()
{
    //Setup Quadric Object
    pObj = gluNewQuadric();
    gluQuadricNormals(pObj, GLU_SMOOTH);

    posx = -5.0f; posy = 0.0f; posz = 2.0f;
    roty = 30.0f;
}

MyMovingSmiley::~MyMovingSmiley()
{
    gluDeleteQuadric(pObj);
}

void MyMovingSmiley::draw()
{
    glPushMatrix();
    glTranslatef(posx, posy, posz);
    glRotatef(roty, 0.0f, 1.0f, 0.0f);

    glPushMatrix();
    //head
    glTranslatef(0.0f, 4.0f, 0.0f);
    glColor3f(1.0f, 1.0f, 0.0f);
}

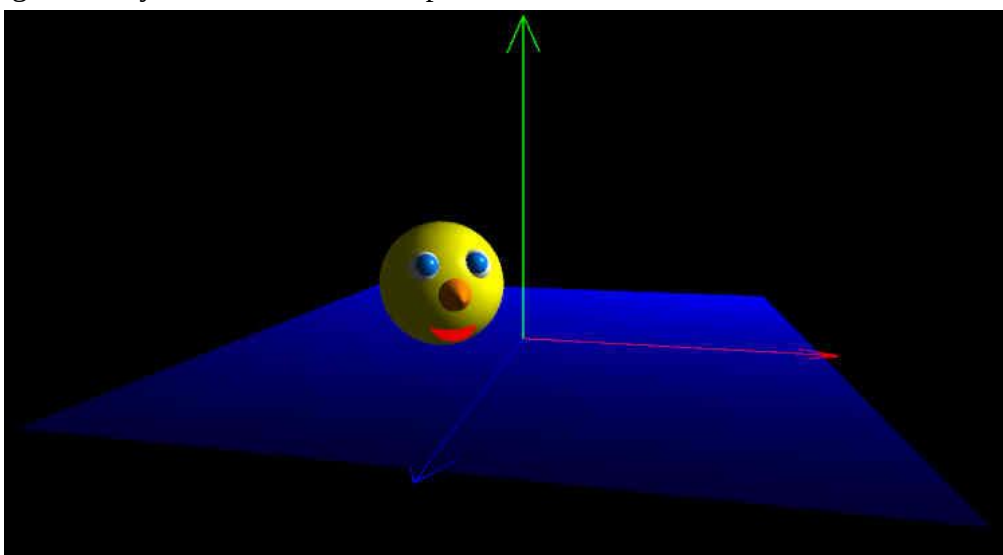
```

```

gluSphere(pObj,4.0f, 20, 20);
//nose
glTranslatef(0.0f, 0.0f, 4.0f);
glColor3f(1.0f, 0.5f, 0.0f);
gluCylinder(pObj,1.0f, 0.0f, 2.0f, 5, 5);
//eyes
glPushMatrix();
    glTranslatef(1.5f, 1.5f, -1.2f);
    glColor3f(1.0f, 1.0f, 1.0f);
    gluSphere(pObj,1.0f, 10, 10);
    glTranslatef(0.0f, 0.2f, 0.5f);
    glColor3f(0.0f, 0.5f, 1.0f);
    gluSphere(pObj,0.7f, 10, 10);
glPopMatrix();
glPushMatrix();
    glTranslatef(-1.5f, 1.5f, -1.2f);
    glColor3f(1.0f, 1.0f, 1.0f);
    gluSphere(pObj,1.0f, 10, 10);
    glTranslatef(0.0f, 0.2f, 0.5f);
    glColor3f(0.0f, 0.5f, 1.0f);
    gluSphere(pObj,0.7f, 10, 10);
glPopMatrix();
//mouth
GLboolean cullingIsOn;
glGetBooleanv(GL_CULL_FACE, &cullingIsOn);
glDisable(GL_CULL_FACE);
glTranslatef(0.0f, -1.8f, -1.2f);
glRotatef(-45, 1.0f, 0.0f, 0.0f);
glColor3f(1.0f, 0.0f, 0.0f);
gluDisk(pObj,0.0f, 1.5f, 10, 10);
if (cullingIsOn==GL_TRUE) glEnable(GL_CULL_FACE);
glPopMatrix();
glPopMatrix();
}

```

Run the program and you should see the output below:



Take note that the position of the “Smiley” has been set at (-5, 0, 2) and the rotation has been set to 30 degrees as indicated by the bold lines in the program.

Your task now is to try to understand how the “smiley” was modelled. Try to understand the portion of the code that has been highlighted above, these codes determine the position and orientation of the “smiley”. Try to change the values of the position and the rotation angle, then run the program again and see how the position and orientation of the “smiley” are being changed.

Task 1B: Moving the Smiley in Straight-Line Movements (20 minutes)

Our next task is to make the “smiley” such that it will move around on its own, in a straight-line manner.

Add the highlighted lines to the MyVirtualWorld class in CGLab09.hpp:

```
class MyMovingSmiley
{
    ...
private:
    ...
    GLfloat velx, vely, velz; //unit per sec
};

class MyVirtualWorld
{
public:
    MyMovingSmiley mymovingsmiley;
    long int timeold, timenew, elapseTime;
    ...
    ...
    void tickTime()
    {
        timenew      = glutGet(GLUT_ELAPSED_TIME) ;
        elapseTime = timenew - timeold;
        timeold      = timenew;

        mymovingsmiley.tickTime(elapseTime) ;
    }

    void init()
    {
        timeold = glutGet(GLUT_ELAPSED_TIME) ;
    }
};
```

Add the highlighted lines to `MyMovingSmiley::MyMovingSmiley()` function in the `CGLab09.cpp` to initialize the starting velocity in x, y, and z directions:

```
MyMovingSmiley::MyMovingSmiley()
{
    //Setup Quadric Object
    pObj = gluNewQuadric();
    gluQuadricNormals(pObj, GLU_SMOOTH);

    posX = -5.0f; posY = 0.0f; posz = 2.0f;
    roty = 30.0f;

    //initial velocity (in unit per second)
    velx = 40.0f;
    vely = 0.0f;
    velz = 30.0f;
}
```

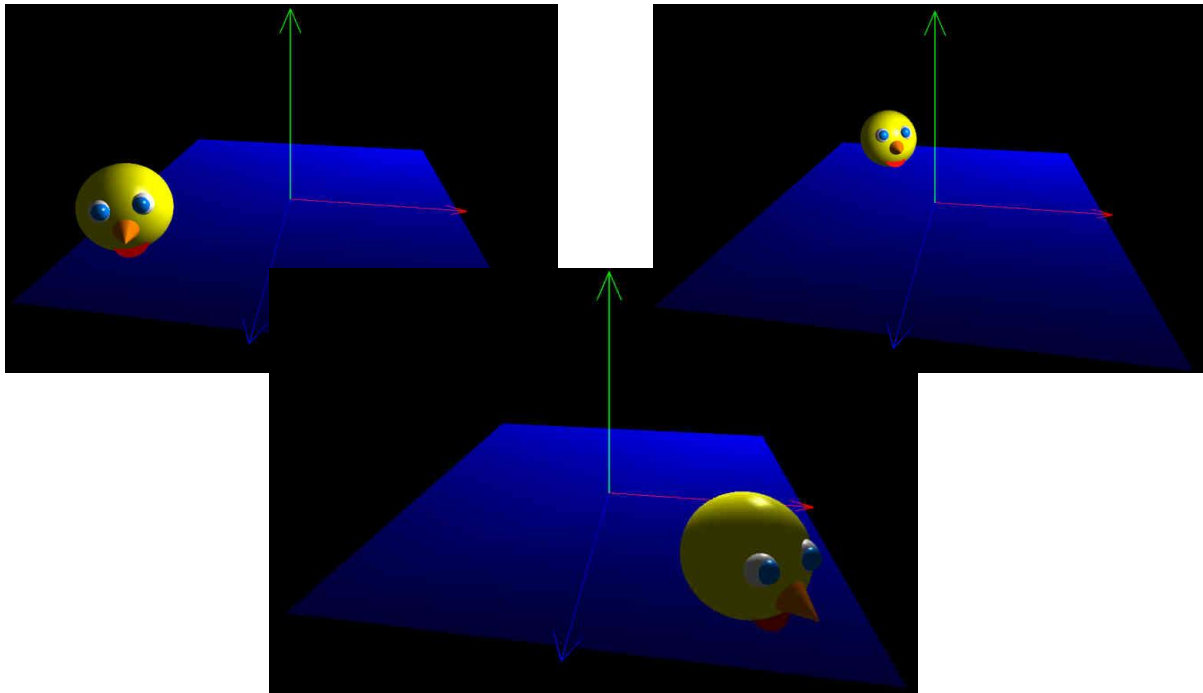
Add the function below to the `CGLab09.cpp`:

```
//elapsed time in milisec
void MyMovingSmiley::tickTime(long int elapsedTime)
{
    float elapsedTimeInSec = elapsedTime / 1000.0f;
    posX += elapsedTimeInSec * velx;
    posY += elapsedTimeInSec * vely;
    posz += elapsedTimeInSec * velz;

    if (posx > 15)
    {
        posX = 30 - posX; //posx = 15 - (posx - 15)
        velx = -velx;
    }
    else if (posx < -15)
    {
        posX = -30 - posX; //posx = -15 - (posx - (-15))
        velx = -velx;
    }

    if (posz > 15)
    {
        posz = 30 - posz;
        velz = -velz;
    }
    else if (posz < -15)
    {
        posz = -30 - posz;
        velz = -velz;
    }
}
```

Run the program and see the animation!



Your task now is to try to understand all the modifications above especially those in the `void MyMovingSmiley::tickTime(...)` function. Take note that the **velocity** is specified as unit per second, while the **elapsed time** is in unit per milliseconds.

The animation occurs following the process below:

- During each frame of display, the `myDisplayFunc(void)` function will call the `tickTime()` function of `MyVirtualWorld` class. In this function, the elapsed time in milliseconds will be calculated by computing the difference between the present time this function is called and the previous time it is called. The time is obtained by calling the GLUT's `glutGet(GLUT_ELAPSED_TIME)` function. This `glutGet(GLUT_ELAPSED_TIME)` function is called the first time in the `init()` function of `MyVirtualWorld`.
- From the `tickTime()` function of `MyVirtualWorld` class, the `tickTime()` function of `MyMovingSmiley` class will be called. This function will compute the **new position** and **orientation** of the “Smiley” based on the elapsed time and with the velocity information provided in the constructor of the `MyMovingSmiley` class.

Task 1C: Make Smiley Rotate and Bounce! (20 minutes)

Modify the program such that the

- “Smiley” will bounce up and down the floor.
- “Smiley” will rotate while it bounces around!

(The additional code needed to do the above is just about a few lines only 😊.)

Task 2: Animation in Hierarchical Modelling (30 minutes)**Task 2A: Preparing the Fan** (15 minutes)

We will try another example that uses hierarchical modelling to add animation to certain moving parts. Add the MyFan class to CGLab09.hpp as below:

```
class MyFan
{
public:
    MyFan();
    ~MyFan();

    void tickTime(long int elapsedTime);
    void draw();

private:
    void drawBase();
    void drawMiddle();
    void drawWings();

    GLUquadricObj *pObj;
    GLfloat pitchangle;
    GLfloat swingangle, wingsangle; //in degree
    GLfloat swingspeed; //in degree per sec
    GLfloat wingspeed; //in degree per sec
};
```

Add the lines to the MyVirtualWorld class in the CGLab09.hpp as highlighted below:

```
class MyVirtualWorld
{
public:
    MyMovingSmiley mymovingsmiley;
    MyFan myfan;
    ...

    void draw()
    {
        drawFloor();
        mymovingsmiley.draw();
        myfan.draw();
    }

    ...
    void tickTime()
    {
        ...
        mymovingsmiley.tickTime(elapsedTime);
        myfan.tickTime(elapsedTime);
    }
    ...
};
```

Add the definition of the functions in MyFan class in CGLab09.cpp as below:

```
MyFan::MyFan()
{
    //Setup Quadric Object
    pObj = gluNewQuadric();
    gluQuadricNormals(pObj, GLU_SMOOTH);
    pitchangle = -20.0f;
    swingangle = 45.0f;
    wingsangle = 30.0f;
    swingspeed = 90.0; //degree per sec
    wingsspeed = 720.0; //degree per sec
}

MyFan::~MyFan()
{
    gluDeleteQuadric(pObj);
}

void MyFan::draw()
{
    glPushMatrix();
    drawBase();
    glTranslatef(0.0f, 10.0f, 0.0f);
    glRotatef(-45, 0.0f, 1.0f, 0.0f);
    glRotatef(swingangle, 0.0f, 1.0f, 0.0f);
    glRotatef(pitchangle, 1.0f, 0.0f, 0.0f);
    glTranslatef(0.0f, 0.0f, 2.0f);
    drawMiddle();
    glTranslatef(0.0f, 0.0f, 6.0f);
    glRotatef(wingsangle, 0.0f, 0.0f, 1.0f);
    drawWings();
    glPopMatrix();
}

void MyFan::drawBase()
{
    glPushMatrix();
    glRotatef(-90.0f, 1.0f, 0.0f, 0.0f);
    glColor3f(1.0f, 0.0f, 0.0f);
    gluCylinder(pObj, 5.0f, 1.0f, 2.0f, 10, 10);
    glColor3f(1.0f, 0.0f, 1.0f);
    gluCylinder(pObj, 1.0f, 1.0f, 10.0f, 10, 10);
    glRotatef(180.0f, 1.0f, 0.0f, 0.0f);
    gluDisk(pObj, 0.0f, 5.0f, 10, 10);
    glPopMatrix();
}

void MyFan::drawMiddle()
{
    glPushMatrix();
    glScalef(0.3f, 0.3f, 1.0f);
    glColor3f(1.0f, 1.0f, 0.0f);
    gluSphere(pObj, 5.0f, 10, 10);
    glTranslatef(0.0f, 0.0f, 5.0f);
    gluCylinder(pObj, 0.5f, 0.5f, 1.5f, 10, 10);
    glPopMatrix();
}
```

```

void MyFan::drawWings()
{
    GLboolean cullingIsOn;
    glGetBooleanv(GL_CULL_FACE, &cullingIsOn);
    glDisable(GL_CULL_FACE);

    GLboolean normalizeIsOn;
    glGetBooleanv(GL_NORMALIZE, &normalizeIsOn);
    glEnable(GL_NORMALIZE);

    glColor3f(0.5f, 1.0f, 0.5f);
    glPushMatrix();
        glBegin(GL_TRIANGLES);
            glNormal3f( 1.0f, 0.0f, 2.0f);
            glVertex3f( 0.0f, 0.0f, 0.0f);
            glVertex3f( 0.0f, 4.0f, 0.0f);
            glVertex3f(-2.0f, 4.0f, 1.0f);

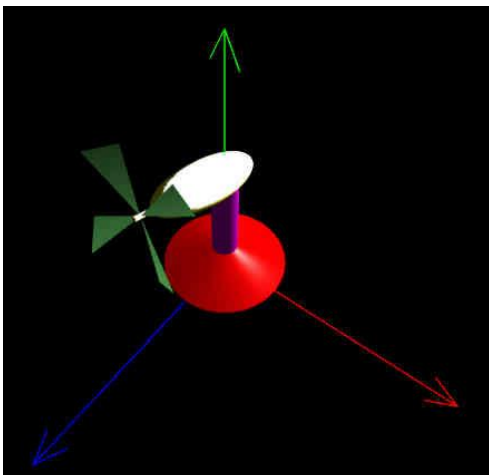
            glNormal3f( 0.0f, 1.0f, 2.0f);
            glVertex3f( 0.0f, 0.0f, 0.0f);
            glVertex3f(-4.0f, 0.0f, 0.0f);
            glVertex3f(-4.0f,-2.0f, 1.0f);

            glNormal3f(-1.0f, 0.0f, 2.0f);
            glVertex3f( 0.0f, 0.0f, 0.0f);
            glVertex3f( 0.0f,-4.0f, 0.0f);
            glVertex3f( 2.0f,-4.0f, 1.0f);

            glNormal3f( 0.0f,-1.0f, 2.0f);
            glVertex3f( 0.0f, 0.0f, 0.0f);
            glVertex3f( 4.0f, 0.0f, 0.0f);
            glVertex3f( 4.0f, 2.0f, 1.0f);
        glEnd();
    glPopMatrix();
    if (cullingIsOn==GL_TRUE) glEnable(GL_CULL_FACE);
    if (normalizeIsOn==GL_TRUE) glEnable(GL_NORMALIZE);
}

void MyFan::tickTime(long int elapsedTime) //elapsed time in milisec
{
    // to be filled
};

```



Run the program and you will get the output shown on the left.

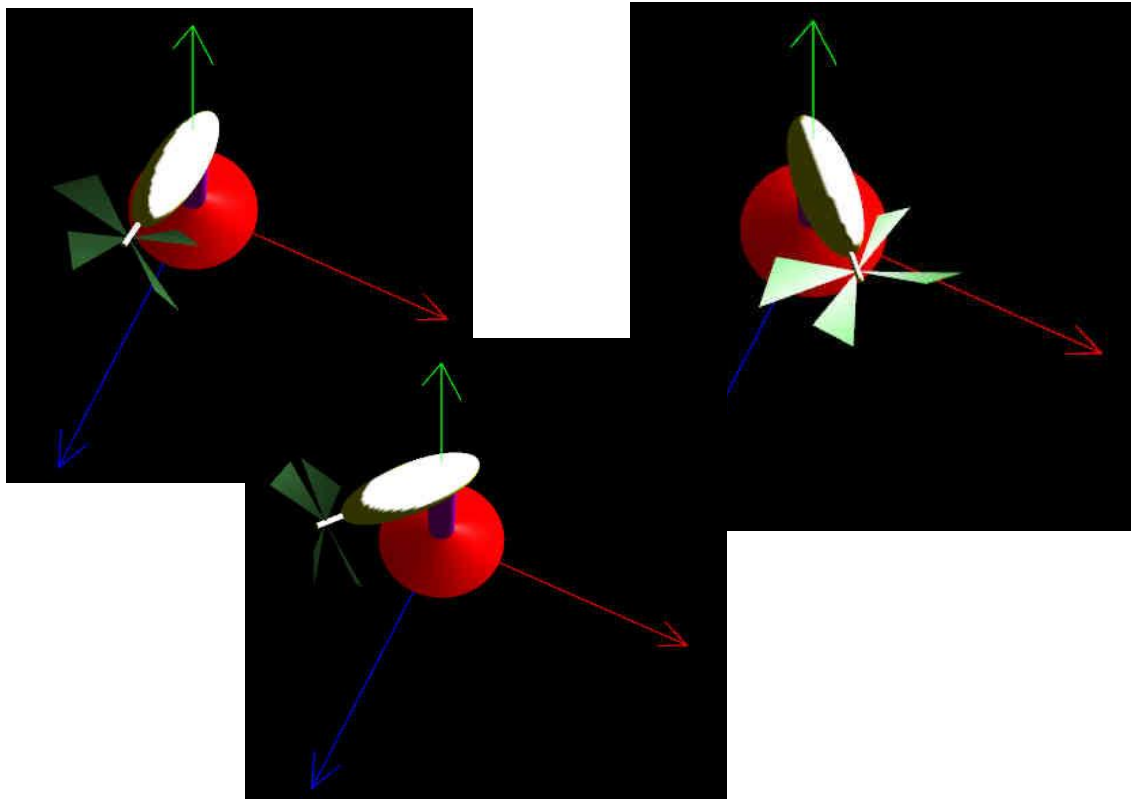
Your task now is to discuss and make sure you understand how this fan is being modelled hierarchically.

Whenever you are doing modelling and animation such as this, you need to be absolutely clear on the coordinate system used so that when you want to incorporate animation, you know exactly what transformation parameters to change in the code.

Task 2B: Make the Fan Rotate and Swing! (15 minutes + Homework)

Complete the `void MyFan::tickTime(...)` function so that the wings of the fan will rotate and the fan will swing from one side to another side.

Sample output:



HAVE FUN AND HOPE TO SEE YOUR ANIMATIONS!!

Extra Exercises

VERY IMPORTANT

BEFORE coming to the next lab, you MUST:

- 1. COMPLETE all the exercises in this lab!**
- 2. DO the extra exercises below (if there are any)!**
- 3. READ the lab instructions!**
- 4. PREPARE files for next lab!**

1. Complete the exercises in this lab if you have not completed them.
2. You probably did not manage to stop the smiley from bouncing “into” the floor, to the space below! Modify your program in **Task 1B** so that the “smiley” will bounce up and down the floor within the box, but not go below the floor.
3. Modify your program in **Task 2B** above so that you can turn on and off the “fan” by pressing a key on the keyboard. Also, modify such that you can enable and disable the swing by pressing a key on the keyboard.
4. Modify your previous labs to make it animated!
5. Be creative, try and create other kinds of animation that looked realistic!

* END OF LAB *